P1018R8
C++ Language Evolution status 🦠 pandemic edition 🦠 2020/11–2021/01

Published Proposal, 2021-01-27

This version:
http://wg21.link/P1018r8

Author:
JF Bastien (Woven Planet)

Audience:
WG21, EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P1018r8.bs

Abstract

This paper is a collection of items that the C++ Language Evolution group has worked on in the latest meeting, their status, and plans for the future.

Table of Contents

§ 1. Executive summary

We have not met in-person since the February 2020 meeting in Prague because of the global pandemic. We're instead holding weekly teleconferences, as detailed in [P2145R1]. We focus on providing non-final guidance, and will use electronic straw polls as detailed in [P2195R0] to move papers and issues forward in a asynchronous manner.

Our main achievements have been:

- **Issue processing**: most of the 50 language evolution issues have proposed resolutions.
- **C++23**: we've started work on papers for C++23 and later.
- **Incubation**: we've acted as EWG-I and "incubated" some early papers by providing early feedback to authors.

This paper outlines the work achieved, other ongoing work, and lists the straw polls that will be submitted for the February 2021 polling period.

§ 2. Paper of note

- [P1000R4] C++ IS schedule
- [P0592R4] To boldly suggest an overall plan for C++23
- [P1999R0] Process: double-check evolutionary material via a Tentatively Ready status
- [P2195R0] Electronic Straw Polls
- [P2145R1] Evolving C++ Remotely

§ 3. Tentatively ready papers

Following our process in [P1999R0], we usually mark papers as tentatively ready for CWG. We would usually take a brief look at the next meeting, and if nothing particular concerns anyone, send them to CWG. However, given the pandemic, we've decided to provide guidance only in virtual teleconferences, and have an asynchronous polling mechanism to officially send papers to CWG or other groups as detailed in [P2195R0]. We will be polling to advance all of these papers in the February 2021 polling period, and therefore avoid listing them twice in this paper.

You can follow the lists of papers on GitHub:

- tentatively ready papers,
- EWG vote on me papers.

§ 4. Issue Processing

We've reviewed 50 Language Evolution issues at the Core groups' request, and have tentative resolutions for most. We don't want to poll all of these at the same time, and will therefore only poll a subset in the February 2021 polling period, reserving other issues for later polling periods. We will therefore only poll the "tentatively ready" issues (since they're tied to papers, and polled with said papers, as outlined below), as well as the "resolved" issues since telecon attendees believe that prior work has already addressed the issues.

§ 7 Remaining Open Issues contains a list of issues which aren't being voted on in this polling period.

§ 5. Polls

The current list of polls which EWG will take in the February 2021 polling period are:

§ 5.1. P2223R1 Trimming whitespaces before line splicing

- [P2223R1]
- GitHub issue
- Notes 2020-10-14
- Not yet seen by WG14, nor the SG22 C / C++ liaison group.
- **Highlight:** make trailing whitespaces after `\` non-significant.
- **Poll:** Forward P2223R1 "Trimming whitespaces before line splicing" to Core, thereby also fixing [CWG1698].

§ 5.2. P2201R0 Mixed string literal concatenation

- [P2201R0]
- GitHub issue
- Notes 2020-08-19
- Seen by WG14 as [N2594], straw Poll: Does the committee wish to adopt N2594 into C23 as is? 18-0-2 passes.
- **Highlight:** string concatenation involving string-literals with *encoding-prefixes* mixing `L""`, `u8""`, `u""`, and `U""` is currently conditionally-supported with implementation-defined behavior, this paper makes it ill-formed.
- **Poll:** Forward P2201R0 "Mixed string literal concatenation" to Core, after adding an Annex C entry.

§ 5.3. P2186R1 Removing Garbage Collection Support

- [P2186R1]
- GitHub issue
- Notes 2020-07-03
- Notes 2020-12-14
- **Highlight:** remove (not deprecate) garbage collection support in C++23.
- **Poll:** Forward P2186R1 "Removing Garbage Collection Support" to Core.

§ 5.4. P2173R0 Attributes on Lambda-Expressions

- [P2173R0]
- GitHub issue
- Notes 2020-04-23
- Notes 2020-07-02
- **Highlight:** allow attributes for lambdas, those attributes appertaining to the function call operator of the lambda.

- **Poll:** Forward P2173R0 "Attributes on Lambda-Expressions" to Core, thereby also fixing [CWG2097].

§ 5.5. P2156R1 Allow Duplicate Attributes

- [P2156R1]
- GitHub issue
- Notes 2020-04-15
- Notes 2020-04-23
- Notes 2020-07-02
- Seen by WG14 as [N2557], straw Poll: Is the committee in favor of adopting N2557 into C23? 19-0-0 passes.
- **Highlight:** current limitations on duplicate attributes are inconsistent, this change removes all limitations on duplicate attributes in an attribute-list for all attributes that specify a limitation. This affects `carries_dependency`, `deprecated`, `fallthrough`, `likely`, `unlikely`, `unused`, `noreturn`, and `no_unique_address`.
- **Poll:** Forward P2156R1 "Allow Duplicate Attributes" to Core, thereby also fixing [CWG1914].

§ 5.6. P2013R3 Freestanding Language: Optional `::operator new`

- [P2013R3]
- GitHub issue
- Notes 2020-02-14
- Notes 2020-08-19
- Not yet seen by WG14, nor the SG22 C / C++ liaison group.
- **Highlight:** on freestanding C++ implementations, make the various default allocating `::operator new` optional. Placement `new` is still required. All `::operator delete` are still required. Hosted implementations are unchanged.
- **Poll:** Forward P2013R3 "Freestanding Language: Optional `::operator new`" to Core.

§ 5.7. P1949R6 C++ Identifier Syntax using Unicode Standard Annex 31

- [P1949R6]
- GitHub issue
- SG16 Unicode group tracking GitHub issue
- C++20 NB comment NL029 05.10
- Notes 2019-12-19
- Notes 2020-02-12
- Notes 2020-02-13
- Notes 2020-02-14
- Notes 2020-06-18
- Notes 2020-09-24
- Not yet seen by WG14, nor the SG22 C / C++ liaison group.
- **Highlight:** C++11 uses a hand-curated list of Unicode codepoints to determine which characters are allowed in identifiers. This is flawed in many ways outlined in the paper. Replace this hand-curated list with the Unicode consortium's recommendations for identifiers [UAX31].
- **Poll:** Forward P1949R6 "C++ Identifier Syntax using Unicode Standard Annex 31" to Core.

§ 5.8. P1938R2 `if consteval`

- [P1938R2]

- GitHub issue
- C++20 NB comment FR22 20.15.10, discussed in Belfast
- Notes 2020-02-14
- Notes 2020-10-08
- **Highlight:** C++20 added `std::is_constant_evaluated()` and `consteval`, which interact poorly with each other. This paper adds a new form of `if` statement to address the poor interaction.
- **Poll:** Forward P1938R2 "`if consteval`" to Core.

§ 5.9. P1847R3 Make declaration order layout mandated

- [P1847R3]
- GitHub issue
- Notes 2019-11-05
- Notes 2020-02-13
- **Highlight:** C++ currently lets compilers reorder data member layout if they have different access controls. Compilers don't take advantage of this permission. This paper removes the unused flexibility.
- **Poll:** Forward P1847R3 "Make declaration order layout mandated" to Core.

§ 5.10. P1401R4 Narrowing contextual conversions to `bool`

- [P1401R4]
- GitHub issue
- Addresses [CWG2320], which was caused by resolving [CWG2039], see Richard Smith's email regarding this approach.
- Notes 2020-02-12
- Notes 2020-09-30
- Notes 2020-10-28
- **Highlight:** allow conversions from integral types to type `bool` in `static_assert` and `if constexpr`-statements.
- **Poll:** Forward P1401R4 "Narrowing contextual conversions to `bool`" to Core.

§ 5.11. P1393R0 A General Property Customization Mechanism

- [P1393R0]
- GitHub issue
- Notes 2020-05-21
- **Highlight:** "properties" are a customization mechanism at the center of [P0443R14] executors. The proposed approach is library-only, but it could be possible to develop a language-based approach instead. The discussion and poll's goal are to avoid a late-changing redesign of executors based on wanting a language-based approach for customization.
- **Poll:** We understand properties and think that specifying them purely in library is the right approach.

§ 5.12. CWG2169 Narrowing conversions and overload resolution

- [CWG2169]

- > Current implementations ignore narrowing conversions during overload resolution, emitting a diagnostic if calling the selected function would involve narrowing. For example:
  >
  > ```
  > struct s { long m };
  > struct ss { short m; };
  > void f( ss );
  > void f( s );
  > void g() {
  >   f({ 1000000 }); // Ambiguous in spite of narrowing for f(ss)
  > }
  > ```
  >
  > However, the current wording of 12.4.3.1.5 [over.ics.list] paragraph 7 says,
  >
  > > Otherwise, if the parameter has an aggregate type which can be initialized from the initializer list according to the rules for aggregate initialization (9.4.1 [dcl.init.aggr]), the implicit conversion sequence is a user-defined conversion sequence with the second standard conversion sequence an identity conversion.
  >
  > In the example above, `ss` cannot be initialized from `{ 1000000 }` because of the narrowing conversion, so presumably `f(ss)` should not be considered. If this is not the intended outcome, paragraph 7 should be restated in terms of having an implicit conversion sequence, as in, e.g., bullet 9.1, instead of a valid initialization.

- Notes 2020-04-29
- JF's email to EWG 2020-10-19, with the follow-up suggestion that no paper was needed and CWG could resolve the issue if given guidance from EWG.
- **Poll:** Guidance to Core: implementations are right, the Standard needs to be fixed to address this issue by following existing implementations.

§ 5.13. CWG2355 Deducing *noexcept-specifiers*

- [CWG2355]
- The list of deducible forms in 13.10.2.5 [temp.deduct.type] paragraph 8 does not include the ability to deduce the value of the constant in a noexcept-specifier, although implementations appear to allow it. Note: multiple standard library implementations rely on it.
- Notes 2020-05-07
- **Poll:** Guidance to Core: modify the Standard such that the value of a constant in a *noexcept-specifier* can be deduced.

§ 5.14. CWG476 Determining the buffer size for placement `new`

- [CWG476]
- Resolved by [CWG2382] (no overhead for placement `new` for arrays). CWG2382 was moved in Belfast as part of [P1969r0].
- Notes 2020-04-09
- **Poll:** Mark CWG476 "Determining the buffer size for placement `new`" as resolved by P1969r0, and a duplicate of CWG2382.

§ 5.15. CWG687 `template` keyword with *unqualified-id*s

- [CWG687]
- Resolved by [P0846r0]
- Notes 2020-04-09
- **Poll:** Mark CWG687 "`template` keyword with *unqualified-id*s" as resolved by P0846r0.

§ 5.16. CWG1326 Deducing an array bound from an *initializer-list*

- [CWG1326]
- Resolved by [P0127R2] Declaring non-type `template` parameters with `auto`, and [CWG1591].
- Notes 2020-04-09
- **Poll:** Mark CWG1326 "Deducing an array bound from an *initializer-list*" as resolved by P0127R2, and a duplicate of CWG1591.

§ 5.17. CWG1331 `const` mismatch with defaulted copy constructor

- [CWG1331]
- Resolved by [P0641r2] (whose title is Resolving Core Issue #1331, also resolves [CWG1426]).
- Notes 2014-06-21
- Notes 2020-04-09
- **Poll:** Mark CWG1331 "`const` mismatch with defaulted copy constructor" as resolved by P0641r2.

§ 5.18. CWG1393 Pack expansions in *using-declaration*s

- [CWG1393]
- Resolved by [P0195r2] (same title as issue)
- Notes 2014-06-21
- Notes 2020-04-09
- **Poll:** Mark CWG1393 "Pack expansions in *using-declaration*s" as resolved by P0195r2.

§ 5.19. CWG1426 Allowing additional parameter types in defaulted functions

- [CWG1426]
- Resolved by [P0641r2] (also resolves [CWG1331])
- Notes 2020-04-09
- **Poll:** Mark CWG1426 "Allowing additional parameter types in defaulted functions" as resolved by P0641r2.

§ 5.20. CWG1561 Aggregates with empty base classes

- [CWG1561]
- Resolved by [P0017r1] (which allowed aggregates to have base classes)
- Notes 2014-06-21
- Notes 2020-04-15
- **Poll:** Mark CWG1561 "Aggregates with empty base classes" as resolved by P0017r1.

§ 5.21. CWG1912 *exception-specification* of defaulted function

- [CWG1912]
- The current rules requiring a defaulted member function to have an exception-specification compatible with that of the implicitly-declared function are overly constraining. It should be possible, for example, to specify that a defaulted move constructor will be non-throwing, based on knowledge available to the programmer, even if the implicitly-declared constructor would be throwing.
- Resolved by [p1286r2]
- Notes 2020-04-23

- **Poll:** Mark CWG1912 "*exception-specification* of defaulted function" as resolved by p1286r2.

5.22. CWG1931 Default-constructible and copy-assignable closure types

- [CWG1931]
- Resolved by [P0624r2] Default constructible and assignable stateless lambdas
- Notes 2020-04-23
- **Poll:** Mark CWG1931 "Default-constructible and copy-assignable closure types" as resolved by P0624r2.

5.23. CWG2295 Aggregates with deleted defaulted constructors

- [CWG2295]
- Should a class with a deleted non-user-provided default constructor be considered an aggregate?
- Resolved by [P1008r1] Prohibit aggregates with user-declared constructors
- Notes 2020-05-07
- **Poll:** Mark CWG2295 "Aggregates with deleted defaulted constructors" as resolved by P1008r1.

5.24. CWG2341 Structured bindings with `static` storage duration

- [CWG2341]
- Resolved by [P1091r3] Extending structured bindings to be more like variable declarations
- Notes 2020-05-07
- **Poll:** Mark CWG2341 "Structured bindings with `static` storage duration" as resolved by P1091r3.

5.25. CWG2343 `void*` non-type template parameters

- [CWG2343]
- According to 13.2 [temp.param] bullet 4.2, non-type template parameters of pointer type must be either pointer to object or pointer to function. This excludes `void*`, which is an object pointer but not a pointer to object. However, most or all current implementations accept `void*` as a non-type template parameter. Notes from the April, 2018 Core teleconference: Not all implementations accept a `void*` template parameter, so this should not be a DR if it is eventually adopted. Furthermore, there is some implementation divergence over the kinds of template arguments that can be passed to a `void*` template parameter.
- Resolved by [P0732R2] (which used `std::strong_ordering` in defining strong structural equality and used that for NTTPs) because [P0515R3] had given `<=>` the type `std::strong_ordering` for "object pointers" (which includes `void*`, even though void is not an object type). Later [P1907R1] replaced all that, saying simply that structural types (which include all scalar types) were acceptable, but that didn't change `void*`'s status. Davis emailed EWG the above text.
- Notes 2020-05-07
- Notes 2020-10-28
- **Poll:** Mark CWG2343 "`void*` non-type template parameters" as resolved by P0732R2.

§ 5.26. CWG??? unqualified lookup in *conversion-type-id*s (Sent by Richard Smith to EWG, from CWG)

- The basic situation looks like this:

  ```
  struct A {
    struct B {};
    operator B();
  };
  void f(A a) { a.operator B(); }
  ```

  There is a special-case lookup rule that allows this code to work: when looking up the `B` in `a.operator B()`, we look inside `a`. Presumably the intent is to allow `operator B` to be named in the same way when it's referenced as it was named when it was declared. This is a fairly general rule: in `<some context>operator B`, we look up the `B` in the same place in which we later look up the name `operator B`.

  Question The issue we are facing is: exactly what lookups does the special-case rule apply to? Consider a few examples:

  ```
  a.operator B<C>();
  a.operator B D::*();
  a.operator decltype(E)();
  ```

  Which of `B`, `C`, `D`, and `E` get the special "look this name up in `a`" rule?

  There is implementation divergence:

  - Clang says none.
  - EDG and MSVC say `B` only.
  - GCC says `B` and `D`.
  - And the consistent rules we're looking at now would say all of `B`, `C`, `D`, and `E` are looked up in `A`.

- Resolved by [P1787R6].
- Notes 2020-10-28
- **Poll:** Mark the CWG issue sent by Richard Smith to EWG on 2020-06 "unqualified lookup in *conversion-type-id*s" as resolved by P1787R6.

For each poll, you will be asked to vote one of:

- Strongly in favor
- In favor
- Neutral
- Against
- Strongly against

You will also have the option to abstain from voting on a particular poll. You will be asked to comment on each poll. This comment is mandatory, as it helps the chair determine consensus.

§ 6. Teleconferences

Here are the minutes for the virtual discussions that were held since the Prague meeting in February 2020:

1. 2020-04-09 Issue Processing
2. 2020-04-15 Issue Processing
3. 2020-04-23 Issue Processing
4. 2020-04-29 Issue Processing
5. 2020-05-07 Issue Processing

6. 2020-05-13 Issue Processing
7. 2020-05-21 A General Property Customization Mechanism—[P1393R0] (P1393 tracking issue)
8. 2020-06-10 Reviewing Deprecated Facilities of C++20 for C++23—[P2139R1] (P2139 tracking issue)
9. 2020-06-18 C++ Identifier Syntax using Unicode Standard Annex 31—[P1949R4] (P1949 tracking issue)
10. 2020-06-24 Extended floating-point types and standard names—[P1467R4] (P1467 tracking issue)
11. 2020-07-02 Allow Duplicate Attributes, Attributes on Lambda-Expressions—[P2156R0] (P2156 tracking issue) and [P2173R0] (P2173 tracking issue)
12. 2020-07-08 Pointer lifetime-end zap and provenance, too—[P1726R3] (P1726 tracking issue)
13. 2020-07-16 Guaranteed copy elision for return variables—[P2025R1] (P2025 tracking issue)
14. 2020-07-30 Reviewing Deprecated Facilities of C++20 for C++23, Removing Garbage Collection Support—[P2139R2] (P2139 tracking issue) and [P2186R0] (P2186 tracking issue)
15. 2020-08-05 Transactional Memory Lite Support in C++—[P1875R0] (P1875 tracking issue)
16. 2020-08-19 Freestanding Language: Optional `::operator new`, Mixed string literal concatenation—[P2013R2] (P2013 tracking issue) and [P2201R0] (P2201 tracking issue)
17. 2020-08-27 Exhaustiveness Checking for Pattern Matching—[P1371R3] (P1371 tracking issue)
18. 2020-09-02 `#embed` - a simple, scannable preprocessor-based resource acquisition method—[P1967R2] (P1967 tracking issue)
19. 2020-09-10 A pipeline-rewrite operator—[P2011R1] (P2011 tracking issue)
20. 2020-09-16 Pattern matching: inspect is always an expression—[P1371R3] (P1371 tracking issue)
21. 2020-09-24 C++ Identifier Syntax using Unicode Standard Annex 31, Member Templates for Local Classes—[P1949R6] (P1949 tracking issue) and [P2044R0] (P2044 tracking issue)
22. 2020-09-30 Narrowing contextual conversions to bool, Generalized pack declaration and usage—[P1401R3] (P1401 tracking issue) and [P1858R2] (P1858 tracking issue)
23. 2020-10-08 Compound Literals, `if consteval`—[P2174R0] (P2174 tracking issue) and [P1938R1] (P1938 tracking issue)
24. 2020-10-14 Inline Namespaces: Fragility Bites, Trimming whitespaces before line splicing—[P1701R1] (P1701 tracking issue) and [P2223R0] (P2223 tracking issue)
25. 2020-10-22 Issues Processing
26. 2020-10-28 Issues Processing
27. 2020-11-05 Deducing `this`—P0847R5 (P0847 tracking issue)
28. 2020-11-19 `goto` in pattern matching
29. 2020-12-03 `auto(x)`: decay-copy in the language—P0849R5 (P0849 tracking issue)
30. 2021-01-14 Polls

§ 7. Remaining Open Issues

The following table lists all remaining open issues referred to EWG by Core or Library. Some of them are ready to be polled but are held back from the February 2021 polling period to limit the number of polls in this round.

From	#	Title	Notes
Core	[CWG2261]	Explicit instantiation of in-class `friend` definition	``` struct S { template <class T> friend void f(T) { } }; template void f(int); // Well-formed? ``` A `friend` is not found by ordinary name lookup until it is explicitly declared in the containing name but declaration matching does not use ordinary name lookup. There is implementation divergence on handling of this example.

Note 2020-04-29 Tentative agreement: This should be well-formed.

SF 1 F 10 N 2 A 1 SA 0

JF emailed EWG / Core about this.

Davis: the current name lookup approach which Core is taking in p1787 would disallow this. Support is possible, it would be inconsistent, but would also be a feature.

Notes 2020-10-22: wait until p1787 is voted into the working draft, because it's making this behavior intentional. At that point, we can vote on marking the issue as Not a Defect. No objection to unanimou consent.

Core	[CWG2270]	Non-`inline` functions and explicit instantiation declarations	[Detailed description pending.]

Hubert: question over the role of the `inline` keyword in relation to explicit instantiation declaratior

For `inline` functions, explicit instantiation declarations do not have the effect of suppressing implic instantiation.

A user's desire for wanting to suppress implicit instantiation can arise for different reasons:

To reduce space in object files, executables, etc. and similarly to reduce the number of input symbols linker

To reduce compile time in performing semantic analysis for instantiations whose definitions are provi elsewhere

To control point-of-instantiation, avoiding contexts where the requisite declarations are not declared

The special rule around inline functions allows `inline`-ness to be used to indicate that the first reaso intent and that instantiation-for-inlining is okay.

Consider the following as a translation unit:

```
template <typename T>
//inline
void f(T t) { g(t); }
enum E : int;
extern template void f(E);
void h(E e) { f(e); }
```

Marking the template definition `inline` would mean that the intended declaration for `g` would need provided as the best candidate at the points-of-instantiation for `f<E>`.

The issue initially points out that this use of the `inline` keyword does not match a view that `inline` essentially an ODR tool to allow multiple definitions (as opposed to a way to indicate desire for inlini proposed that `extern template` merely has the effect of suppressing definitions in terms of linkag (regardless of the `inline` keyword). Such a change would affect the usability of the feature for user that falls within the latter two options above.

I am not sure if CWG is asking EWG a specific question other than the general "we do not believe thi wording or obvious consistency issue; is this an issue in terms of design?"

Meeting:

Hubert forked the thread on the reflector. Might want the education SG to take a look, or might want a

Meeting 2020-10-28: Inbal will try to put together the wording, to prove / disprove whether this is a c

Lib	[LWG2432]	initializer_list assignability	std::initializer_list::operator= [support.initlist] is horribly broken and it needs deprecation: std::initializer_list<foo> a = {{1}, {2}, {3}}; a = {{4}, {5}, {6}}; // New sequence is already destroyed. Assignability of initializer_list isn't explicitly specified, but most implementations supply a default assignment operator. I'm not sure what [description] says, but it probably doesn't matter. **Proposed resolution:** Edit [support.initlist] p1, class template initializer_list synopsis, as indicated: namespace std { template<class E> class initializer_list { public: […] constexpr initializer_list() noexcept; initializer_list(const initializer_list&) = default; initializer_list(initializer_list&&) = default; initializer_list& operator=(const initializer_list&) = delete; initializer_list& operator=(initializer_list&&) = delete; constexpr size_t size() const noexcept; […] }; […] } LWG telecon appears to want a language change to disallow assigning a braced-init-list to an std::initializer_list but still permit move assignment of std::initializer_list objects. That is, auto il1 = {1,2,3}; auto il2 = {4,5,6}; il1 = {7,8,9}; // currently well-formed but dangles immediately; should be ill-formed il1 = std::move(il2); // currently well-formed and should remain so

Meeting: Proposed resolution:

```
initializer_list(const initializer_list&) = default;
initializer_list(initializer_list&&) = default;
[[deprecated]] initializer_list& operator=(const initializer_list&) = defa
[[deprecated]] initializer_list& operator=(initializer_list&&) = default;
```

SF F N A SA

0 3 12 0 0

JF emailed LEWG, to see if they have an opinion, no feedback. Asked LEWG chairs to schedule for a telecon.

LWG discussed priority.

Lib [LWG2813] std::function should not return dangling references

If a std::function has a reference as a return type, and that reference binds to a prvalue returned by the that it wraps, then the reference is always dangling. Because any use of such a reference results in unc behaviour, the std::function should not be allowed to be initialized with such a callable. Instead, the pr should be ill-formed.

A minimal example of well-formed code under the current standard that exhibits this issue:

int main() {

std::function<const int&()> F([]{ return 42; });

int x = F(); // oops!

}

Proposed resolution:

Add a second paragraph to the remarks section of 20.14.16.2.1 [func.wrap.func.con]:

template<class F> function(F f);

-7- Requires: F shall be CopyConstructible.

-8- Remarks: This constructor template shall not participate in overload resolution unless

- F is Lvalue-Callable (20.14.16.2 [func.wrap.func]) for argument types ArgTypes... and return typ and
- If R is type "reference to T" and INVOKE(ArgTypes...) has value category V and type U:
  - V is a prvalue, U is a class type, and T is not reference-related (9.4.3 [dcl.init.ref]) to U, and
  - V is an lvalue or xvalue, and either U is a class type or T is reference-related to U.

[…]

Tim: LWG in Batavia 2018 would like a way to detect when the initialization of a reference would bi a temporary. This requires compiler support, since there's no known way in the current language to do

reliably in the presence of user-defined conversions (see thread starting at https://lists.isocpp.org/lib/2017/07/3256.php).

Meeting:

Tim wrote p2255 to address this.

Ville thinks there should be an analysis of alternative approaches.

Also see P0932.

Core [CWG728] Restrictions on local classes

Now that the restriction against local classes being used as template arguments has been lifted, they ar useful, yet they are still crippled. For some reason or oversight, the restriction against local classes be templates or having member templates was not lifted. Allowing local classes to have member templat facilitates generic programming (the reason for lifting the other restriction), especially when it comes visitor-pattern (see the boost::variant documentation and the following example) as implemented in b the boost::MPL library (since functors have to be template classes in mpl, and higher-order functors h have member templates to be useful). A local class with a member template would allow this desirabl solution:

```
#include <boost/variant.hpp>

int main() {

  struct times_two_generic: public boost::static_visitor<> {

    template <typename T> void operator()(T& operand) const {

        operand += operand;

    }

  };

  std::vector<boost::variant<int, std::string>> vec;

  vec.push_back(21);

  vec.push_back("hello ");

  times_two_generic visitor;

  std::for_each(vec.begin(), vec.end(), boost::apply_visitor(visitor));

}
```

Is there any compelling reason not to allow this code? Is there any compelling reason not to allow loc classes to be templates, have friends, or be able to define their static data members at function scope? Wouldn't this symmetry amongst local and non-local classes make the language more appealing and l embarrassing?

Also see p2044, seen by EWGI in Prague and requested to be merged with P1988, and implementors

The current paper draft addresses all of this.

Meeting: (notes)

Some folks unsure that we should do anything here. Revisit the resolution with P2044.

Extension issues are a closed status on the issues list, like NAD Future was for LWG I find it helpful t a status that says EWG may still be considering it.

Suggest closing as Not A Defect because we have significant implementation concerns, but we'll expl design space in P2044. All in favor.

Core	[CWG794]	Base-derived conversion in member type of pointer-to-member conversion	Related to CWG 170, drafting by Clark seems unlikely. This is section 2.1 of Jeff Snyder's P0149R0, was approved by EWG, 4 years ago, waiting for wording. JF reached out to Jeff. Did wording with Jens, main blocker is lack of implementation. **Meeting:** (notes) Suggest closing as Not A Defect because we have implementation uncertainties, but explore the design space in P0149. ABI group will discuss. All in favor.
Core	[CWG900]	Lifetime of temporaries in range-based for	// some function std::vector<int> foo(); // correct usage auto v = foo(); for(auto i : reverse(v)) { std::cout << i << std::endl; } // problematic usage for(auto i : reverse(foo())) { std::cout << i << std::endl; } **Meeting:** (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect because it's a c which might have effects on existing code (might cause bugs), and might need to change more than ju range-based loops. See p0614, p0577, p0936, p1179. We'll explore the design space in a separate pap circled back with Nico on this, might write a paper. All in favor.
Core	[CWG944]	reinterpret_cast for all types with the same size and alignment	During the discussion of issue 799, which specified the result of using reinterpret_cast to convert an o to its own type, it was observed that it is probably reasonable to allow reinterpret_cast between any tw that have the same size and alignment. Additional note, April, 2015: It has been suggested that this question may more properly be the provir EWG, especially in light of discussions during the resolution of issue 330. The priority has been delet allow CWG to reconsider its status. Rationale (May, 2015): CWG agreed that this question should be considered from a language design perspective and is thus being referred to EWG. Suggest rejection: we chose to add std::bit_cast to provide exactly this functionality instead of extendi reinterpret_cast. **Meeting:** (notes) Suggest closing as Not A Defect. bit_cast addresses some of the usecases this issue i cover. We'd want separate papers to support other usescases, but have nothing outstanding at the mom in favor.
Core	[CWG1008]	Querying the alignment of an object	https://godbolt.org/z/_SN8iF • GCC already implements this extension without issuing a warning • Clang and EDG implement this extension for gcc compatibility, with a warning • MSVC does not yet implement this feature Quick example using 'auto' illustrates why we might want this capability for objects as well as types. Principle of least astonishment suggests it is surprising for sizeof and alignof to behave differently in regard. Recommend shipping this straight to core as soon as we can find a wording champion. We need to discuss with WG14. Questions for EWG to answer before forwarding:

			• Should this be a unary operator, like sizeof, so "alignof x" is valid? Or should it be like typeid, w parens are required? • What would this mean? Is it the alignment of the type of the object? Or the compiler's best guess alignment of the expression itself? Should it take into account any facts that are known about the expression other than its type? If so, which ones? (For example, if applied to an expression x or e x is declared with an `alignas` attribute, is that value returned?) This needs a design paper rather than going straight to core. https://godbolt.org/z/TeVA9T GCC seems to report the alignment of the object not just of decltype(ob **Meeting:** (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect, the design is c especially around alignment of object versus type. Invite a paper, Inbal will pitch in, Alidair can collal All in favor. Inbal's paper: P2152R0
Core	[CWG1077]	Explicit specializations in non-containing namespaces	The current wording of 9.8.1.2 [namespace.memdef] and 13.9.3 [temp.expl.spec] requires that an exp specialization be declared either in the same namespace as the template or in an enclosing namespace would be convenient to relax that requirement and allow the specialization to be declared in a non-enc namespace to which one or more if the template arguments belongs. Additional note, April, 2015: See EWG issue 48. Might allow us to revert DR2061 and all the horribleness that created and described in p1701. The pr 1077 addresses is the motivating factor in dr2061. Also see CWG 2370. **Meeting:** (notes) Suggest closing as Not A Defect. See p0665, minutes. Continue under p0665 or a fo version of it. All in favor.
Core	[CWG1433]	*trailing-return-type* and point of declaration	template <class T> T list(T x); template <class H, class ...T> auto list(H h, T ...args) -> decltype(list(args...)); // list isn't in scope in its own *trailing-return-type* auto list3 = list(1, 2, 3); **Meeting:** (notes, also discussed in Rapperswil 2014) there might be compiler divergence according to Daveed. Suggest closing as Not A Defect, it would be tricky to change behavior without ambiguity. "I this would break existing code that relies on seeing only previous declarations. No objection to unanir consent.
Core	[CWG1451]	Objects with no linkage in non-type template arguments	According to 13.4.2 [temp.arg.nontype] paragraph 1 bullet 3, only objects with linkage can be used to non-type template arguments. Is this restriction still needed? It would be convenient to use block-scop objects as template arguments. Resolved by N4268. The following is valid: template <int *> struct A; auto f() { static int x; A<&x> *ap = nullptr; return ap; }

			There is no longer any linkage restriction. There is a restriction on storage duration, which seems reas (you can use the address of a local static variable but not of a local automatic variable as a non-type te argument). **Meeting:** (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect. Addressed by I No objection to unanimous consent.
Core	[CWG1469]	Omitted bound in array *new-expression*	The syntax for noptr-new-declarator in 7.6.2.7 [expr.new] paragraph 1 requires an expression, even th the bound could be inferred from a braced-init-list initializer. It is not clear whether 9.4.1 [dcl.init.agg paragraph 4, An array of unknown size initialized with a brace-enclosed initializer-list containing n in clauses, where n shall be greater than zero, is defined as having n elements (9.3.3.4 [dcl.array]). should be considered to apply to the new-type-id variant, e.g., new (int[]){1, 2, 3} This was addressed by p1009 **Meeting:** (notes) Suggest closing as Not A Defect. Addressed by P1009. No objection to unanimous c
Core	[CWG1555]	Language linkage and function type compatibility	Currently function types with different language linkage are not compatible, and 7.6.1.2 [expr.call] pa 1 makes it undefined behavior to call a function via a type with a different language linkage. These fe are generally not enforced by most current implementations (although some do) between functions wi and C++ language linkage. Should these restrictions be relaxed, perhaps as conditionally-supported be Somewhat related to CWG1463. **Meeting:** (notes) no strong consensus at the moment, Erich Keane brought this up with SG12 Undefi Behavior. Long discussion, will need to revisit, need a paper. **Meeting Oct 22nd 2020:** will need a volunteer to write a paper, but the wording in the standard is unambiguous, and we have existence proof of platforms which use different calling conventions betw and C++. However many major compilers have chosen to ignore this. Not a defect, but we welcome a to change the status quo. No objection to unanimous consent.
Core	[CWG1643]	Default arguments for template parameter packs	Although 13.2 [temp.param] paragraph 9 forbids default arguments for template parameter packs, allo them would make some program patterns easier to write. Should this restriction be removed? **Meeting:** (notes) Suggest closing as Not A Defect. Interesting design space, but needs a paper, see N3 objection to unanimous consent.
Core	[CWG1864]	List-initialization of array objects	The resolution of issue 1467 now allows for initialization of aggregate classes from an object of the sa type. Similar treatment should be afforded to array aggregates. See recent discussion on allowing 'auto x[] = {1, 2, 3};' -- this topic came up there. The two questions answered independently, but some consider them to be related. **Meeting:** (notes) Suggest closing as Not A Defect. Fixing anything in this space would require a pape considers the entire design space, Timur might be interested in this. No objection to unanimous conse
Core	[CWG1871]	Non-identifier characters in *ud-suffix*	JF forwarded to SG16 Unicode given their work on TR31. Tracked on GitHub. SG16 reviewed D194 still needs wording changes. Discussed at the April 22nd SG16 telecon. **SG16 Poll:** Is there any objection to unanimous consent for recommending rejection of this proposal? objection to unanimous consent. **EWG Meeting:** (notes 2020-04-15, notes 2020-05-07) Suggest closing as Not A Defect. No objection unanimous consent.

Core	[CWG1876]	Preventing explicit specialization	A desire has been expressed for a mechanism to prevent explicitly specializing a given class template, particular std::initializer_list and perhaps some others in the standard library. It is not clear whether si adding a prohibition to the description of the templates in the library clauses would be sufficient or wl core language mechanism is required. **Meeting:** (notes) Suggest closing as Not A Defect. No objection to unanimous consent. This could be language feature, would need library usecase examples. **Poll:** we are interested in such a paper SF 2 F 13 N 6 A 1 SA 0
Core	[CWG1915]	Potentially-invoked destructors in non-throwing constructors	Since the base class constructor is non-throwing, the deleted base class destructor need not be referen There's a typo in the issues list here: it should say "Since the *derived* class constructor is non-throwin deleted base class destructor need not be referenced." **Meeting:** (notes 2020-04-23) the proposed wording changes the implementation leeway in two-phase unwinding, which breaks some existing ABIs. We would need a paper to explore this further. Suggest as Not A Defect. No objection to unanimous consent.
Core	[CWG1923]	Lvalues of type void	There does not seem to be any significant technical obstacle to allowing a void* pointer to be derefere and that would avoid having to use weighty circumlocutions when casting to a reference to an object designated by such a pointer. Might consider this a duplicate of the discussion on "regular void". JF reached out to Matt. He has an implementation in clang. Needs to update the paper. Might need a v to present. Daveed would be interested in presenting. **Meeting:** (notes 2020-04-23) Suggest closing as Not A Defect. Explore under the "regular void" umb No objection to unanimous consent.
Core	[CWG1934]	Relaxing *exception-specification* compatibility requirements	According to 14.5 [except.spec] paragraph 4, If any declaration of a function has an exception-specification that is not a noexcept-specification allo exceptions, all declarations, including the definition and any explicit specialization, of that function sh have a compatible exception-specification. This seems excessive for explicit specializations, considering that paragraph 6 applies a looser require for virtual functions: If a virtual function has an exception-specification, all declarations, including the definition, of any fu that overrides that virtual function in any derived class shall only allow exceptions that are allowed by exception-specification of the base class virtual function. The rule in paragraph 3 is also problematic in regard to explicit specializations of destructors and defa special member functions, as the implicit exception-specification of the template member function can determined. There is also a related problem with defaulted special member functions and exception-specifications According to 9.5.2 [dcl.fct.def.default] paragraph 3,

			If a function that is explicitly defaulted has an explicit exception-specification that is not compatible ([except.spec]) with the exception-specification on the implicit declaration, then • if the function is explicitly defaulted on its first declaration, it is defined as deleted; • otherwise, the program is ill-formed. This rule precludes defaulting a virtual base class destructor or copy/move functions if the derived cla function will throw an exception not allowed by the implicit base class member function. **Meeting:** (notes 2020-04-23) JF will work with Mike to update wording to C++20. Will revisit. From Mike: This issue is NAD since we eliminated typed exception-specifications. The current wordi dealing only with noexcept(true) and noexcept(false), does not have this issue. Will remove “extensio status. **Meeting:** no objection to CWG taking it back, and marking it as NAD.
Core	[CWG1957]	decltype(auto) with direct-list-initialization	Paper N3922 changed the rules for deduction from a braced-init-list containing a single expression in initialization context. Should a corresponding change be made for decltype(auto)? E.g., auto x8a = { 1 }; // decltype(x8a) is std::initializer_list<int> decltype(auto) x8d = { 1 }; // ill-formed, a braced-init-list is not an expression auto x9a{ 1 }; // decltype(x9a) is int decltype(auto) x9d{ 1 }; // decltype(x9d) is int See also issue 1467, which also effectively ignores braces around a single expression, this change wou parallel to that one, even though the primary motivation for delctype(auto) is in the return type of a forwarding function, where direct-initialization does not apply. **Meeting:** (notes 2020-04-23) Suggest closing as Not A Defect. This would be a language change, it’s that we want to make such a change. It would require a paper. Mike Spertus is writing a paper with so overlap, will cover this as well. No objection to unanimous consent.
Core	[CWG2111]	Array temporaries in reference binding	The current wording of the Standard appears to permit code like `void f(const char (&)[10]);` `void g() {` `  f("123");` `  f({'a','b','c','\0'});` `}` creating a temporary array of ten elements and binding the parameter reference to it. This is controver should be reconsidered. (See issues 1058 and 1232.) **Meeting:** (notes 2020-04-23) JF digging more, talking to Richard about this. Somewhat related to P2174 compound literals. Would be very strange if `const char (&&x)[10] = {'a', 'b', 'c'};` ... were invalid but ... `const char x[10] = {'a', 'b', 'c'};`

			`const char (&&x)[3] = {'a', 'b', 'c'};` ... were both OK. Maybe either we should allow the trailing elements to be zeroed in general (the statu or not (a major breaking change). Which means we should reject the issue on consistency grounds. Th general rule is that if T &&r = init; ... can't bind directly, we create a temporary initialized as if with T r = init; ... and bind to that. (And similarly for const references.) Another question: Do we want to support (T){inits} as a synonym for T{inits}? **Meeting Oct 22nd 2020:** Note that the issue itself is defective, and `f("123")` isn't valid. CWG2352 references p1358, and was resolved, leading us to believe that this is now mainstream and controversial. Not a defect. No objection to unanimous consent.
Core	[CWG2125]	Copy elision and comma operator	Currently, _N4750_.15.8 [class.copy] paragraphs 31-32 apply only to the name of a local variable in determining whether a return expression is a candidate for copy elision or move construction. Would i sense to extend that to include the right operand of a comma operator? X f() { X x; return (0, x); } **Meeting:** (notes 2020-04-23) Consider expanding to other places that expand bit-field-ness such as re throw : x;. Suggest closing as Not A Defect. Will need a paper to address, no current volunteer for this objection to unanimous consent.
Core	[CWG2132]	Deprecated default generated copy constructors	EWG has indicated that they are not currently in favor of removing the implicitly declared defaulted c constructors and assignment operators that are eprecated in _N4750_.15.8 [class.copy] paragraphs 7 a Should this deprecation be removed? Related: discussing under p2139 deprecations. **Meeting:** (note 2020-04-29) Suggest closing as Not A Defect. No objection to unanimous consent. W want to remove entirely, or un-deprecate. This will need a paper, Ville will talk with Alisdair about for the topic from p2139.
Core	[CWG914]	Value-initialization of array types	Although value-initialization is defined for array types and the () initializer is permitted in a mem-init naming an array member of a class, the syntax T() (where is an array type) is explicitly forbidden by 7 [expr.type.conv] paragraph 2. This is inconsistent and the syntax should be permitted. Rationale (July, 2009): The CWG was not convinced of the utility of this extension, especially in ligh questions about handling the lifetime of temporary arrays. This suggestion needs a proposal and analy the EWG before it can be considered by the CWG. This has become a more severe inconsistency after we adopted Ville's P0960 for C++20. Now it's not that has this weird special-case restriction, it's (a, b, c) too:

using X = int[];

X x{1, 2, 3}; // ok, int[3]

X y(1, 2, 3); // ok, int[3]

f(X{1, 2, 3}); // ok, int[3] temporary

f(X(1, 2, 3)); // error

Meeting: (notes) it is a defect, need a paper. David Stone trying to find a volunteer to write said pape favor.

Core	[CWG1463]	extern "C" alias templates	Currently 13 [temp] paragraph 4 forbids any template from having C linkage. Should alias templates l exempt from this prohibition, since they do not have any linkage? Additional note, April, 2013: It was suggested (see messages 23364 through 23367) that relaxing this restriction for alias templates could provide a way of addressing the long-standing lack of a way of sp a language linkage for a dependent function type (see issue 13). The priority was deleted to allow CW consider the implications of that potential application of the facility. We should either have some way to express a dependent function type with C language linkage (and s accept 1555 below) or we should remove the notion that C language linkage (or not) is part of the fun type at all. (Apparently some targets used it; are they still in use? EDG probably knows.) Actively discussed on CWG reflector. **Davis' interpretation of CWG discussion**: I don't speak for Core, of course, but my (Evolutionary) thoughts are those given (last) in the message which you replied: neither the wording nor the apparent intent of [temp.pre]/6's restrictions on C linka templates make any sense. Since templates (and explicit specializations) can't have C linkage of the r mangling variety (which the standard calls language linkage of a (function) name) but can have C link the calling-convention variety (which the standard calls language linkage of a (function) type), extern should grant them the latter and not the former with no error. This has certain obvious applications in C APIs with callbacks. (Put differently, CWG13 shouldn't have been rejected; it appears to have gott bogged down in questions of syntax, but we have an adequate syntactic workaround that we merely h permit.) CWG1463 asks for a (very) proper subset of the above: merely that extern "C" be allowed to apply to templates (claiming incorrectly that they have no name with linkage at all). I consider it more relevan more productive) to talk about whether entities have names with language linkage than with the ordin kind. The Core reflector discussion (which seems to have finished for the moment) also touched on the case "the same" function template is declared with two different language linkages for its type, but the only relevant Evolution input there would be a decision to go the opposite way and forbid function templat having types with C language linkage entirely. (They currently can, but it's mostly or entirely useless If (for CWG1555, also on your list) the rules about language linkage of (function) types are sufficient relaxed, then CWG1463 may be moot (and my extension of it with it), but that seems unlikely given I recent identification of a case where they matter. **Meeting:** (notes 2020-04-15, notes 2020-10-22, also discussed in Rapperswil 2014) We agree that thi issue. extern "C" on a template should only affect calling convention, and not the mangling. Davis an Hubert volunteer to write a paper. Unanimous consent.

Core	[CWG1790]	Ellipsis following function parameter pack	Although the current wording permits an ellipsis to immediately follow a function parameter pack, it i clear that the <cstdarg> facilities permit access to the ellipsis arguments. The problem here (which is not explained in the issue) is: how do you supply the name of the last para before the ellipsis to va_start? You can't put the name of a pack there (it wouldn't be expanded) and tl no way to name the last element of the pack (nor to deal with the case where the pack is empty). **Meeting:** (notes) 3 options: fix wording around "last parameter", remove facility entirely (either va_s function declarator), try to invent a language facility. JF emailed EWG to see if anyone has a strong preference, or if we should send back to CWG to fix wording, long discussion. **Michael Spertus:** I am willing to commit to including and analysis on this in an upcoming paper on parameter packs. JF followed up with Michael and Barry, no response.
Core	[CWG1962]	Type of __func__	Two questions have arisen regarding the treatment of the type of the __func__ built-in variable. First, implementations accept void f() { typedef decltype(__func__) T; T x = __func__; } even though T is specified to be an array type. In a related question, it was noted that __func__ is implicitly required to be unique in each function, a not only the value but the type of __func__ are implementation-defined; e.g., in something like inline auto f() { return &__func__; } the function type is implementation-specific. These concerns could be addressed by making the value prvalue of type const char* instead of an array lvalue. Rationale (November, 2018): See also issue 2362, which asks for the ability to use __func__ in a cons function. These two goals are incompatible. The deep question here is about __func__ and the ODR. Does EWG want implementations to someho behave as if __func__ is the same in all copies of an inline function (in which case it can have an arra and be usable in constant expressions, but the *demangling* algorithm used to construct it becomes part ABI), or does EWG want implementations to behave as if __func__ may differ between copies, so is not known until runtime (in which case it must have either pointer or incomplete array type, and its va not usable in constant expressions -- but its address could still be usable). Note: C++20 has std::source_location::function_name. **Meeting:** (notes 2020-04-23) We should discuss this with WG14. This is indeed a language issue. No objection to unanimous consent. We'll need a paper, potentially considering what Reflection could do brought it up on the mailing list. We probably need a paper to disentangle this.
Core	[CWG2228]	Ambiguity resolution for cast to function type	Proposed resolution. C++ has a blanket disambiguation rule that says if a sequence of tokens can be interpreted as either a name or an expression, the type-name interpretation is chosen. This is unhelpful in cases like

(T())+3

where the current rules make "(T())" a cast to the type "function with no parameters returning T" rathe parenthesized value-initialization of a temporary of type T. The former interpretation is always ill-forı you can't cast to a function type - while the latter could be useful.

Richard's proposed resolution, cited above, is to avoid the ambiguity by changing the grammar so tha like "(T())" cannot be parsed as a cast. The wording in the proposal applies that change to a number o different contexts where the ambiguity can come into play. There are two contexts where the change i applied, however: 1) the operand of typeid, and 2) a template-argument. During our discussion yester there was some support for the idea of applying the change to typeid, as well, although that would be breaking change for any programs that rely on the current disambiguation to get type information for function types. There was general agreement, however, to exclude template arguments, since they are for things like std::function.

CWG would, therefore, like some guidance from EWG on two questions. First, should we apply the n syntax to the operand of typeid, even though it's a breaking change?

More generally, the question was raised whether we should make this change at all. Although resolvir ambiguity in the other direction would arguably be more convenient in many cases, there is a tension that convenience and the complexity of the language. In particular, we would be creating a situation w the exact same sequence of tokens would mean two different things, depending on the context in whic appear. Is the convenience worth the cost in complexity?

Meeting: (note 2020-04-29, notes 2020-03-23, note from Core summer 2020, notes from Core Pragu notes from Core 2019-01-07, notes from Core Kona 2019, notes from Core Cologne 2019) This is an we'd like to change the standard to resolve it: SF 0 F 6 N 4 A 3 SA 2

Davis emailed EWG reflector. Long discussion.

Core [CWG2296] Are default argument instantiation failures in the "immediate context"?

Example 1:

```
template <typename T, typename U = T>
void fun(T v, U u = U()) {}
void fun(...) {}
struct X {
  X(int) {} // no default ctor
};
int main()  {  fun (X(1)); }
```

Consider the following example (taken from issue 3 of paper P0348R0):

Example 2:

```
template <typename U> void fun(U u = U());
```

```
struct X {

  X(int) {}

};

template <class T> decltype(fun<T>()) g(int) { }

template <class> void g(long) { }

int main() { g<X>(0); }
```

When is the substitution into the return type done? The current specification makes this example ill-fo
because the failure to instantiate the default argument in the decltype operand is not in the immediate
of the substitution, although a plausible argument for making this a SFINAE case can be made.

Meeting: (notes 2020-05-07)

The first example under issue 3 of paper P0348R0 should become well-formed.

SF F N A SA

0 1 4 3 6

The second example under issue 3 of paper P0348R0 should become well-formed.

SF F N A SA

1 5 7 3 0

This is an issue. We'd like to see a paper addressing it. It should explore what "Immediate context" m
No objection to unanimous consent.

Daveed / Hubert might entertain writing a paper explaining this.

Ville emailed EWG about this.

JF contacted Andrzej to see if he's interested in addressing this. Not sure he is.

Meeting 2020-10-28: Tomasz will write a paper which exposes the issue and what he thinks should b
(but not resolving wording itself). Hubert remembers an email about this, will find it and sync with JF

Meeting 2021-01-14: Andrzej wrote a paper for this (emailed 2021-01-12 to EWG).

Core	[CWG2362]	__func__ should be constexpr	The definition of __func__ in 9.5.1 [dcl.fct.def.general] paragraph 8 is: static const char __func__[] = "function-name"; This prohibits its use in constant expressions, e.g., int main () { // error: the value of __func__ is not usable in a constant expression constexpr char c = __func__[0]; } Rationale (November, 2018): See also issue 1962, which asks that the type of __func__ be const char two goals are incompatible. **Meeting:** handle with 1962.

§ 8. Near-future EWG plans

We will continue to work on issue resolution and C++23, prioritizing according to [P0592R4].

§ References

§ Informative References

[CWG1008]
Steve Clamage. Querying the alignment of an object. 27 November 2009. extension. URL: https://wg21.link/cwg1008

[CWG1077]
Mike Spertus. Explicit specializations in non-containing namespaces. 13 June 2010. extension. URL: https://wg21.link/cwg1077

[CWG1326]
Faisal Vali. Deducing an array bound from an initializer-list. 28 May 2011. extension. URL: https://wg21.link/cwg1326

[CWG1331]
Daniel Krügler. const mismatch with defaulted copy constructor. 15 June 2011. extension. URL: https://wg21.link/cwg1331

[CWG1393]
Daniel Krügler. Pack expansions in using-declarations. 10 September 2011. extension. URL: https://wg21.link/cwg1393

[CWG1426]
Nikolay Ivchenkov. Allowing additional parameter types in defaulted functions. 8 December 2011. extension. URL: https://wg21.link/cwg1426

[CWG1433]
Jason Merrill. trailing-return-type and point of declaration. 20 December 2011. extension. URL: https://wg21.link/cwg1433

[CWG1451]
Daniel Krügler. Objects with no linkage in non-type template arguments. 1 February 2012. extension. URL: https://wg21.link/cwg1451

[CWG1463]
Daveed Vandevoorde. extern "C" alias templates. 19 August 2011. extension. URL: https://wg21.link/cwg1463

[CWG1469]
Johannes Schaub. Omitted bound in array new-expression. 12 February 2012. extension. URL: https://wg21.link/cwg1469

[CWG1555]
Daniel Krügler. Language linkage and function type compatibility. 18 September 2012. extension. URL: https://wg21.link/cwg1555

[CWG1561]
Gabriel Dos Reis. Aggregates with empty base classes. 29 September 2012. extension. URL: https://wg21.link/cwg1561

[CWG1591]
Peter Dimov. Deducing array bound and element type from initializer list. 1 December 2012. CD4. URL: https://wg21.link/cwg1591

[CWG1643]
Vinny Romano. Default arguments for template parameter packs. 17 March 2013. extension. URL: https://wg21.link/cwg1643

[CWG1698]
David Krauss. Files ending in \. 10 June 2013. open. URL: https://wg21.link/cwg1698

[CWG1790]
Daryle Walker. Ellipsis following function parameter pack. 1 October 2013. extension. URL: https://wg21.link/cwg1790

[CWG1864]
Alisdair Meredith. List-initialization of array objects. 15 February 2014. extension. URL: https://wg21.link/cwg1864

[CWG1871]
Richard Smith. Non-identifier characters in ud-suffix. 17 February 2014. extension. URL: https://wg21.link/cwg1871

[CWG1876]
John Spicer. Preventing explicit specialization. 19 February 2014. extension. URL: https://wg21.link/cwg1876

[CWG1912]
Ville Voutilainen. exception-specification of defaulted function. 10 April 2014. extension. URL: https://wg21.link/cwg1912

[CWG1914]
Aaron Ballman. Duplicate standard attributes. 15 April 2014. extension. URL: https://wg21.link/cwg1914

[CWG1915]
Aaron Ballman. Potentially-invoked destructors in non-throwing constructors. 15 April 2014. extension. URL: https://wg21.link/cwg1915

[CWG1923]
Richard Smith. Lvalues of type void. 6 May 2014. extension. URL: https://wg21.link/cwg1923

[CWG1931]
Ryou Ezoe. Default-constructible and copy-assignable closure types. 24 May 2014. extension. URL: https://wg21.link/cwg1931

[CWG1934]
Vinny Romano. Relaxing exception-specification compatibility requirements. 3 June 2014. extension. URL: https://wg21.link/cwg1934

[CWG1957]
Vinny Romano. decltype(auto) with direct-list-initialization. 20140630. extension. URL: https://wg21.link/cwg1957

[CWG1962]
Steve Clamage. Type of __func__. 4 July 2014. drafting. URL: https://wg21.link/cwg1962

[CWG2039]
Richard Smith. Constant conversions to bool. 9 November 2014. CD4. URL: https://wg21.link/cwg2039

[CWG2097]
Jonathan Caves. Lambdas and noreturn attribute. 12 March 2015. extension. URL: https://wg21.link/cwg2097

[CWG2111]
Vinny Romano. Array temporaries in reference binding. 2 April 2015. extension. URL: https://wg21.link/cwg2111

[CWG2125]
Vinny Romano. Copy elision and comma operator. 6 May 2015. extension. URL: https://wg21.link/cwg2125

[CWG2132]
Nico Josuttis. Deprecated default generated copy constructors. 28 May 2015. extension. URL: https://wg21.link/cwg2132

[CWG2169]
David Krauss. Narrowing conversions and overload resolution. 26 August 2015. extension. URL: https://wg21.link/cwg2169

[CWG2228]
Richard Smith. Ambiguity resolution for cast to function type. 2 February 2016. drafting. URL: https://wg21.link/cwg2228

[CWG2261]
Jens Maurer. Explicit instantiation of in-class friend definition. 28 April 2016. extension. URL: https://wg21.link/cwg2261

[CWG2270]
Richard Smith. Non-inline functions and explicit instantiation declarations. 10 June 2016. extension. URL: https://wg21.link/cwg2270

[CWG2295]
Thomas Köppe. Aggregates with deleted defaulted constructors. 24 June 2016. extension. URL: https://wg21.link/cwg2295

[CWG2296]
Jason Merrill. Are default argument instantiation failures in the “immediate context”?. 25 June 2016. extension. URL: https://wg21.link/cwg2296

[CWG2320]
Richard Smith. constexpr if and boolean conversions. 6 September 2016. extension. URL: https://wg21.link/cwg2320

[CWG2341]
John Spicer. Structured bindings with static storage duration. 4 April 2017. extension. URL: https://wg21.link/cwg2341

[CWG2343]
Daveed Vandevoorde. void* non-type template parameters. 17 April 2017. extension. URL: https://wg21.link/cwg2343

[CWG2355]
John Spicer. Deducing noexcept-specifiers. 6 September 2017. extension. URL: https://wg21.link/cwg2355

[CWG2362]
Anthony Polukhin. __func__ should be constexpr. 23 October 2017. open. URL: https://wg21.link/cwg2362

[CWG2382]
Paul Sanders. Array allocation overhead for non-allocating placement new. 16 July 2018. review. URL: https://wg21.link/cwg2382

[CWG476]
Ben Hutchings. Determining the buffer size for placement new. 14 September 2004. extension. URL: https://wg21.link/cwg476

[CWG687]
Mihai Rusu. template keyword with unqualified-ids. 27 February 2008. extension. URL: https://wg21.link/cwg687

[CWG728]
Faisal Vali. Restrictions on local classes. 5 October 2008. extension. URL: https://wg21.link/cwg728

[CWG794]
CH. Base-derived conversion in member type of pointer-to-member conversion. 3 March 2009. extension. URL: https://wg21.link/cwg794

[CWG900]
Thomas J. Gritzan. Lifetime of temporaries in range-based for. 12 May 2009. extension. URL: https://wg21.link/cwg900

[CWG914]
Gabriel Dos Reis. Value-initialization of array types. 10 June 2009. extension. URL: https://wg21.link/cwg914

[CWG944]
Miller. reinterpret_cast for all types with the same size and alignment. 15 July 2009. extension. URL: https://wg21.link/cwg944

[LWG2432]
David Krauss. initializer_list assignability. EWG. URL: https://wg21.link/lwg2432

[LWG2813]
Brian Bi. std::function should not return dangling references. EWG. URL: https://wg21.link/lwg2813

[N2557]
Melanie Blower. Allow Duplicate Attributes. 01 September 2020. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2557.pdf

[N2594]
JeanHeyd Meneide. Mixed Wide String Literal Concatenation. 26 October 2020. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2594.htm

[P0017r1]
Oleg Smolsky. Extension to aggregate initialization. 24 October 2015. URL: https://wg21.link/p0017r1

[P0127R2]
James Touton, Mike Spertus. Declaring non-type template arguments with auto. 23 June 2016. URL: https://wg21.link/p0127r2

[P0195r2]
Robert Haberlach, Richard Smith. Pack expansions in using-declarations. 8 November 2016. URL: https://wg21.link/p0195r2

[P0443R14]
Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, H. Carter Edwards, Gordon Brown, D. S. Hollman. A Unified Executors Proposal for C++. 15 September 2020. URL: https://wg21.link/p0443r14

[P0515R3]
Herb Sutter, Jens Maurer, Walter E. Brown. Consistent comparison. 10 November 2017. URL: https://wg21.link/p0515r3

[P0592R4]
Ville Voutilainen. To boldly suggest an overall plan for C++23. 25 November 2019. URL: https://wg21.link/p0592r4

[P0624r2]
Louis Dionne. Default constructible and assignable stateless lambdas. 10 November 2017. URL: https://wg21.link/p0624r2

[P0641r2]
Daniel Krügler, Botond Ballo. Resolving Core Issue #1331 (const mismatch with defaulted copy constructor). 8 November 2017. URL: https://wg21.link/p0641r2

[P0732R2]
Jeff Snyder, Louis Dionne. Class Types in Non-Type Template Parameters. 6 June 2018. URL: https://wg21.link/p0732r2

[P0846r0]
John Spicer. ADL and Function Templates that are not Visible. 8 November 2017. URL: https://wg21.link/p0846r0

[P1000R4]
Herb Sutter. C++ IS schedule. 14 February 2020. URL: https://wg21.link/p1000r4

[P1008r1]
Timur Doumler, Arthur O'Dwyer, Richard Smith, Howard E. Hinnant, Nicolai Josuttis. Prohibit aggregates with user-declared constructors. 8 June 2018. URL: https://wg21.link/p1008r1

[P1091r3]
Nicolas Lesser. Extending structured bindings to be more like variable declarations. 18 February 2019. URL: https://wg21.link/p1091r3

[P1286R2]
Richard Smith. Contra CWG DR1778. 18 February 2019. URL: https://wg21.link/p1286r2

[P1371R3]
Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. Pattern Matching. 15 September 2020. URL: https://wg21.link/p1371r3

[P1393R0]
D. S. Hollman, Chris Kohlhoff, Bryce Lelbach, Jared Hoberock, Gordon Brown, Michał Dominiak. A General Property Customization Mechanism. 13 January 2019. URL: https://wg21.link/p1393r0

[P1401R3]
Andrzej Krzemieński. Narrowing contextual conversions to bool. 27 May 2020. URL: https://wg21.link/p1401r3

[P1401R4]
Andrzej Krzemieński. Narrowing contextual conversions to bool. 8 October 2020. URL: https://wg21.link/p1401r4

[P1467R4]
David Olsen, Michał Dominiak. Extended floating-point types and standard names. 14 June 2020. URL: https://wg21.link/p1467r4

[P1701R1]
Nathan Sidwell. Inline Namespaces: Fragility Bites. 13 September 2020. URL: https://wg21.link/p1701r1

[P1726R3]
Paul E. McKenney, Maged Michael, Jens Mauer, Peter Sewell, Martin Uecker, Hans Boehm, Hubert Tong, Niall Douglas, Will Deacon, Michael Wong, and David Goldblatt. Pointer lifetime-end zap. 21 February 2020. URL: https://wg21.link/p1726r3

[P1787R6]
S. Davis Herring. Declarations and where to find them. 28 October 2020. URL: https://wg21.link/p1787r6

[P1847R3]
Pal Balog. Make declaration order layout mandated. 1 March 2020. URL: https://wg21.link/p1847r3

[P1858R2]
Barry Revzin. Generalized pack declaration and usage. 1 March 2020. URL: https://wg21.link/p1858r2

[P1875R0]
Michael Spear, Hans Boehm, Victor Luchangco, Michael Scott, Michael Wong. Transactional Memory Lite Support in C++. 7 October 2019. URL: https://wg21.link/p1875r0

[P1907R1]
Jens Maurer. Inconsistencies with non-type template parameters. 8 November 2019. URL: https://wg21.link/p1907r1

[P1938R1]
Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. if consteval. 2 March 2020. URL: https://wg21.link/p1938r1

[P1938R2]
Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. if consteval. 14 October 2020. URL: https://wg21.link/p1938r2

[P1949R4]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. C++ Identifier Syntax using Unicode Standard Annex 31. 5 June 2020. URL: https://wg21.link/p1949r4

[P1949R6]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. C++ Identifier Syntax using Unicode Standard Annex 31. 15 September 2020. URL: https://wg21.link/p1949r6

[P1967R2]
JeanHeyd Meneide. #embed - a simple, scannable preprocessor-based resource acquisition method. 2 March 2020. URL: https://wg21.link/p1967r2

[P1969r0]
William M. (Mike) Miller. Core Language Working Group "ready" issues for the November, 2019 (Belfast) meeting. 8 November 2019. URL: https://wg21.link/p1969r0

[P1999R0]
Ville Voutilainen. Process proposal: double-check evolutionary material via a Tentatively Ready status. 25 November 2019. URL: https://wg21.link/p1999r0

[P2011R1]
Barry Revzin, Colby Pike. A pipeline-rewrite operator. 16 April 2020. URL: https://wg21.link/p2011r1

[P2013R2]
Ben Craig. Freestanding Language: Optional ::operator new. 14 August 2020. URL: https://wg21.link/p2013r2

[P2013R3]
Ben Craig. Freestanding Language: Optional ::operator new. 13 September 2020. URL: https://wg21.link/p2013r3

[P2025R1]
Anton Zhilin. Guaranteed copy elision for return variables. 15 June 2020. URL: https://wg21.link/p2025r1

[P2044R0]
Robert Leahy. Member Templates for Local Classes. 12 January 2020. URL: https://wg21.link/p2044r0

[P2139R1]
Alisdair Meredith. Reviewing Deprecated Facilities of C++20 for C++23. 15 June 2020. URL: https://wg21.link/p2139r1

[P2139R2]
Alisdair Meredith. Reviewing Deprecated Facilities of C++20 for C++23. 15 July 2020. URL: https://wg21.link/p2139r2

[P2145R1]
Bryce Adelstein Lelbach, Titus Winters, Fabio Fracassi, Billy Baker, Nevin Liber, JF Bastien, David Stone, Botond Ballo, Tom Honermann. Evolving C++ Remotely. 15 September 2020. URL: https://wg21.link/p2145r1

[P2156R0]
Erich Keane. Allow Duplicate Attributes. 15 April 2020. URL: https://wg21.link/p2156r0

[P2156R1]
Erich Keane. Allow Duplicate Attributes. 2 July 2020. URL: https://wg21.link/p2156r1

[P2173R0]
Daveed Vandevoorde, Inbal Levi, Ville Voutilainen. Attributes on Lambda-Expressions. 15 May 2020. URL: https://wg21.link/p2173r0

[P2174R0]
Zhihao Yuan. Compound Literals. 16 May 2020. URL: https://wg21.link/p2174r0

[P2186R0]
JF Bastien, Alisdair Meredith. Removing Garbage Collection Support. 12 July 2020. URL: https://wg21.link/p2186r0

[P2186R1]
JF Bastien; Alisdair Meredith. Removing Garbage Collection Support. 14 December 2020. URL: https://wg21.link/p2186r1

[P2195R0]
Bryce Adelstein Lelbach. Electronic Straw Polls. 15 September 2020. URL: https://wg21.link/p2195r0

[P2201R0]
Jens Maurer. Mixed string literal concatenation. 14 July 2020. URL: https://wg21.link/p2201r0

[P2223R0]
Corentin Jabot. Trimming whitespaces before line splicing. 14 September 2020. URL: https://wg21.link/p2223r0

[P2223R1]
Corentin Jabot. Trimming whitespaces before line splicing. 17 October 2020. URL: https://wg21.link/p2223r1

[UAX31]
Mark Davis. Unicode Identifier and Pattern Syntax. 13 February 2020. Unicode Standard Annex #31. URL: https://www.unicode.org/reports/tr31/tr31-33.html